

Tool-Calling Agents using LangChain and Gemini — Healthcare

Objective

The objective of this assignment is to help learners understand how **tool-calling agents** can be designed for real-world healthcare scenarios using:

- LangChain
- Gemini
- Tool Calling
- Custom Python Tools
- Agentic Decision Making
- System Prompts and Guardrails
- Natural Language Interaction

Learners will develop an **AI-powered Healthcare Triage Assistant** where Gemini acts as the reasoning engine and uses a custom Python tool whenever structured patient information needs to be evaluated.

The implementation should follow the same basic architecture demonstrated in the reference notebook:

User Query → Gemini Agent → Tool Selection → Tool Execution → Gemini Response

No external dataset is required. Learners should create the required synthetic healthcare information directly inside the notebook using Python dictionaries/lists.

USE CASE: Healthcare Patient Triage Assistant

Business Context

A hospital wants to build an AI assistant that can help its front-desk and care-coordination teams perform an initial **patient's urgency assessment**.

The assistant should be able to:

- Understand the patient's symptoms and basic information.
- Identify the relevant medical category.
- Use a custom tool to calculate an initial severity score.
- Highlight important risk indicators.
- Provide an appropriate recommendation.
- Identify cases where **human medical intervention is required immediately**.

The system is **not intended to diagnose patients**. Its purpose is to demonstrate how an AI agent can use a structured tool to support an initial triage workflow.

Step 1 — Create Synthetic Healthcare Data

Create a Python dictionary containing synthetic information for different symptom categories.

For example:

```
symptom_category_keywords = {  
    "respiratory": [  
        "cough",  
        "breathing difficulty",  
        "shortness of breath",  
        "chest congestion"  
    ],  
  
    "cardiac": [  
        "chest pain",  
        "palpitations",  
        "heart rate",  
        "dizziness"  
    ],  
  
    "infection": [  
        "fever",  
        "chills",  
        "body ache",  
        "fatigue"  
    ]  
}
```

You may define additional categories if required.

Also create a synthetic severity/risk mapping that the tool can use.

For example:

```
severity_weights = {  
    "breathing difficulty": 30,  
    "chest pain": 35,  
    "high fever": 20,
```

```
"dizziness": 15,  
"mild cough": 5  
}
```

The data should remain completely synthetic and should be defined inside the notebook.

Step 2 — Create a Healthcare Assessment Tool

Create a custom LangChain tool using the @tool decorator.

The tool should accept:

- Patient symptoms
- Patient age
- Relevant basic information

The tool should:

1. Analyze the symptoms provided.
2. Identify matching risk indicators.
3. Calculate an initial severity score.
4. Identify missing/unknown risk indicators.
5. Return the assessment in a structured JSON format.

For example, the tool could return:

```
{  
  "severity_score": 75,  
  "risk_level": "HIGH",  
  "matched_indicators": [  
    "chest pain",  
    "breathing difficulty"  
  ],  
  "missing_indicators": [],  
  "human_intervention_required": true  
}
```

The exact scoring methodology is left to the learner.

Step 3 — Create the Gemini Agent

Use Gemini as the reasoning engine for the healthcare assistant.

The agent should have access to the healthcare assessment tool.

The system prompt should establish appropriate guardrails.

The assistant should be instructed to:

- Use the tool whenever structured symptom assessment is required.
- Keep responses short and structured.
- Clearly explain the assessment.
- Never present the result as a definitive medical diagnosis.
- Immediately highlight situations requiring human medical attention.
- Base the response on the information provided by the user.

Step 4 — Build the Agent Workflow

Create an agent that accepts a natural-language patient query.

For example:

Patient is 62 years old and is experiencing chest pain,
breathing difficulty and dizziness since this morning.

The agent should determine that the healthcare assessment tool is relevant, invoke the tool, receive the structured result, and then generate a human-readable response.

The expected interaction should conceptually be:

Patient Query

↓

Gemini Agent

↓

Healthcare Assessment Tool

↓

Severity / Risk Assessment

↓

Gemini

↓

Final Recommendation

Step 5 — Test the Agent

Create at least **three test cases**.

Test Case 1 — Low-Risk Scenario

Example:

Patient is 25 years old and has had a mild cough for two days with no breathing difficulty or chest pain.

The agent should identify the situation as relatively low risk and provide an appropriate recommendation.

Test Case 2 — Moderate-Risk Scenario

Example:

Patient is 48 years old and has fever, fatigue and persistent cough for four days.

The agent should identify the relevant risk indicators and recommend appropriate follow-up.

Test Case 3 — High-Risk Scenario

Example:

Patient is 67 years old and is experiencing chest pain, breathing difficulty and dizziness.

The agent should identify the case as requiring **immediate human medical attention**.

Step 6 — Extend the Use Case

Extend the healthcare agent with the following scenarios:

Scenario A — Irrelevant Symptom Category

Provide symptoms that do not clearly match any predefined category.

The agent should:

- Recognize the limitation.
- Avoid making unsupported conclusions.
- Highlight the need for human assessment where appropriate.

Scenario B — Incomplete Patient Information

Provide a query where important information such as age or symptom duration is missing.

The agent should identify the missing information instead of assuming values.

Scenario C — Multiple Risk Indicators

Provide a patient query containing symptoms from multiple categories.

The agent should use the tool appropriately and explain which indicators contributed to the risk assessment.

Expected Learning Outcome

By completing this assignment, learners should be able to demonstrate how:

- A Python function can be converted into an LLM-accessible tool.
- Gemini can decide when a tool should be invoked.
- Structured tool output can be incorporated into an LLM response.
- System prompts can establish agent behavior and guardrails.
- Agents can handle multiple real-world scenarios using the same tool.
- Human intervention can be explicitly identified by an AI agent.